LifeLaw: A Jurisdiction-Aware, Retrieval-Only Legal Intelligence Platform for Hallucination-Free Question Answering

Kaustubha Venkata Eluri
Northeastern University
eluri.k@northeastern.edu

***Abstract*—Consumer-facing legal information tools increasingly rely on large language models (LLMs) to answer user questions, but free-text generation in a legal-advice-adjacent domain introduces a serious hallucination risk: a fabricated citation or an invented statutory claim can mislead a lay user with no way to verify it. This paper presents LifeLaw, a personalized legal-intelligence platform that builds a jurisdiction-filtered feed of live federal and state legislation, a rights library, court case summaries, and a question-answering (Q&A) engine that is deliberately *retrieval-only*: answers are assembled from structured fields attached to retrieved documents rather than generated as free text by an LLM. We describe the system architecture – a Next.js 15 frontend, a FastAPI backend, and a PostgreSQL database extended with pgvector for cosine-similarity search over locally embedded rights topics, statutes, and bills – and the jurisdiction-filtering pipeline that integrates the Congress.gov and LegiScan APIs. We document the key design decisions that shape the system, including the removal of a local sentence-transformers/PyTorch inference path in favor of a hosted Hugging Face embedding API, an anonymous device-identity model, and per-source failure isolation in the feed pipeline. We report that no formal precision/recall evaluation of the retrieval engine has been conducted to date, and we describe what such an evaluation would require. We conclude with the system's limitations and directions for future work.**

***Index Terms*—retrieval-augmented systems, hallucination mitigation, legal informatics, pgvector, question answering, jurisdiction filtering**

I. Introduction

Large language models have made it trivial to build a chat-style interface over any corpus of text, and legal information is an obvious target: statutes, case law, and legislative bills are text-heavy, jargon-laden, and poorly indexed for lay readers. The temptation is to let an LLM read a user's question, search a corpus, and generate a free-text answer. In most domains this is an acceptable trade-off. In a legal-advice-adjacent domain it is not: a hallucinated case citation, an invented statutory deadline, or a fabricated procedural requirement can cause real harm to a user who has no independent way to check the claim, and who may reasonably assume that a polished, fluent answer is accurate simply because it reads that way.

LifeLaw is built around the premise that the two most valuable properties an educational (not advisory) legal tool can offer a lay user are (1) relevance – surfacing only the bills, rights topics, and case law that actually pertain to the user's state, city, county, and personal situation (e.g., tenant, gig worker, H-1B visa holder), and (2) verifiability – every answer must be traceable to a specific underlying document, with no synthesis step capable of introducing content that was not actually retrieved. The system therefore combines a jurisdiction-filtered feed of live federal and state bills (via the Congress.gov and LegiScan APIs), a curated rights library, court case summaries, and a Q&A engine that performs retrieval over a pgvector-indexed corpus and assembles its answer from the retrieved document's structured fields, explicitly avoiding an LLM call in the answer-generation path.

This paper documents the architecture and design rationale of LifeLaw as an engineering artifact: what was built, why the retrieval-only design was chosen over generative synthesis, how jurisdiction filtering works end to end, and what is and is not known about the system's retrieval quality at this stage.

II. Related Work

Retrieval-augmented generation (RAG) is now a standard architecture for grounding LLM outputs in an external corpus: a retriever selects relevant passages from a document store, and a generator conditions its output on those passages, with the goal of producing answers that are both more accurate and more attributable than closed-book generation [1]. The motivation for RAG is precisely the hallucination problem that motivates LifeLaw: LLMs asked to answer from parametric memory alone will confidently produce plausible-sounding but false claims, and conditioning generation on retrieved evidence reduces (without eliminating) this failure mode, since the generation step can still deviate from, misstate, or extrapolate beyond what was actually retrieved.

LifeLaw departs from the standard RAG pattern at exactly this point. Rather than using retrieved passages as conditioning context for a generative model, LifeLaw removes the generative step from the answer path entirely: retrieval identifies the most relevant document via vector similarity, and the "answer" is a structured assembly of that document's pre-authored fields (e.g., topic summary, applicable law, plain-English explanation, source excerpt, confidence score). This can be understood as an extreme, deliberately conservative point on the RAG design spectrum – retrieval with zero generation – traded off against the fluency and flexibility that free-text generation would otherwise provide. The system also layers regex-based fast paths for common, high-frequency

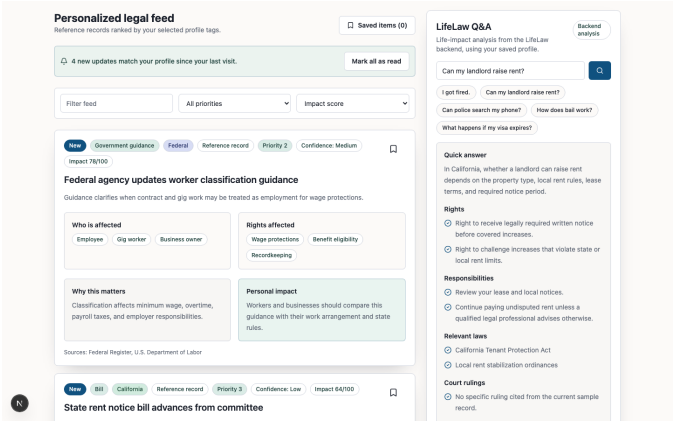

Fig. 1. The personalized jurisdiction feed, rendered from the user's state, city, county, industry, and profile tags.

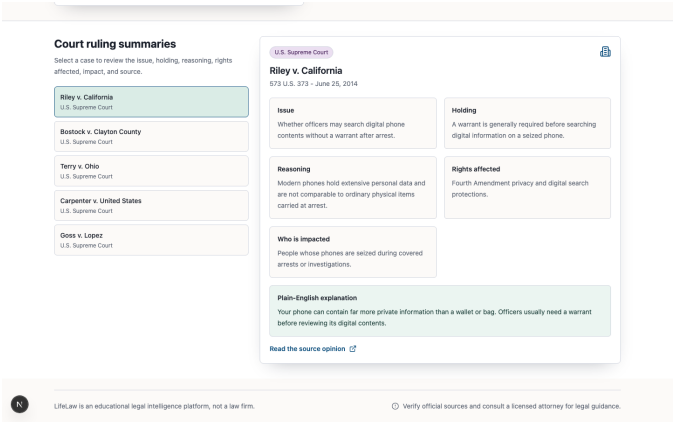

Fig. 2. Court case summaries surfaced for the user's jurisdiction, presented alongside the source excerpt and confidence indicators used to make retrieval provenance visible.

question patterns ahead of the vector search, a lightweight complement to semantic retrieval for queries with predictable phrasing.

III. System Architecture

LifeLaw is a three-tier system: a Next.js 15 / React 19 frontend (TypeScript, strict mode), a FastAPI backend, and a PostgreSQL database extended with the pgvector extension. The frontend drives state from a single top-level profile object (state, city, county, industry, and profile tags such as tenant or H-1B) and renders it through section-level components; all data access goes through a typed API client (`lib/api.ts`) against backend endpoints for the feed, Q&A, rights, search, cases, and profiles (`/api/feed`, `/api/qa`, `/api/rights`, `/api/search`, `/api/cases`, `/api/profiles`). Figure 1 shows the personalized jurisdiction feed generated from a user's profile, and Figure 2 shows the court case summary view surfaced by the same profile-driven pipeline.

The core technical contribution of the system is the design of the `/api/qa` path, which is retrieval-only rather than generative. The pipeline is:

1) **Embedding.** User questions and the curated corpus of rights topics, statutes, and legal documents are embedded with `all-MiniLM-L6-v2` (384 dimensions). Embedding is computed via the free-tier Hugging Face Inference API rather than a locally hosted model.
2) **Vector search.** The corpus embeddings are stored in a PostgreSQL table (`legal_sources`) using pgvector, and a question embedding is matched against the corpus using cosine-distance nearest-neighbor search (`retrieval.py`).
3) **Structured field assembly, not generation.** The top match's pre-authored structured fields are assembled into the response – there is no LLM call in the answer path. If no candidate clears a similarity threshold, the system reports that nothing matched rather than producing a best-effort guess.
4) **Fast paths.** A layer of regex-based pattern matching handles common phrasings (e.g., "I got fired," "police searched my phone") ahead of the vector search step, for latency and precision on frequent query shapes.
5) **Show your work.** Every answer surfaces the matched source document, an excerpt from it, and a confidence score, so the user can verify the basis for the answer rather than trust it on the strength of its prose.

This design eliminates what the system's own documentation calls the "hallucination surface": because no generative model ever produces the answer text, there is no step in the pipeline capable of inventing a fact, a citation, or a legal claim that does not already exist verbatim in the underlying corpus. The cost of this choice is that answers are constrained to the vocabulary and coverage of the curated corpus – the system cannot paraphrase, synthesize across multiple documents, or answer a question that falls outside the topics that have been manually curated and embedded via the `app.services.ingest` pipeline.

A secondary but load-bearing architectural decision was moving embedding computation off the backend process itself. The original implementation used a local `sentence-transformers`/PyTorch inference path, which out-of-memory-crashed on a 512MB Render instance; embedding computation was moved to the hosted Hugging Face Inference API to remove the memory-heavy runtime dependency from the deployed backend.

IV. Jurisdiction Filtering Pipeline

Every feed item, bill, and rights topic in LifeLaw is filtered against a user's declared jurisdiction (state, city, county) and profile tags, rather than presenting an undifferentiated national feed. The ingestion side of this pipeline has two independent sources, each isolated behind its own failure boundary:

- **Federal bills** are pulled from the Congress.gov `/v3/bill` endpoint, sorted by most recently updated, and auto-tagged against profile categories (Tenant, Employee, H-1B, Gig worker, etc.) so that a bill can be

matched to a user's profile tags without the user having to read it first.

- **State bills** are pulled from LegiScan's `getMasterList` endpoint, which covers all 50 states and Washington, D.C., sorted by last action date.

The `/api/feed` endpoint wraps each of these two external API calls in its own try/except block, so that a failure or outage in one upstream source (e.g., LegiScan being unreachable) degrades the feed to federal-only rather than breaking the endpoint entirely – a per-source failure isolation strategy rather than an all-or-nothing external dependency. New items since the user's last visit are tracked and badged as "New" in the UI ("Radar"), and jurisdiction scope (federal / state / court / local) is rendered with distinct design-token colors so a user can tell at a glance what level of government a given item pertains to. Untrusted upstream content – bill titles and summaries sourced from external APIs – is HTML-escaped before it is placed into outbound content such as the weekly digest email, since it originates outside the system's trust boundary.

V. Design Decisions

The project's documented design decisions, and their rationale, are as follows:

- **No LLM call in the Q&A path.** Retrieval-only synthesis was chosen deliberately over generative synthesis specifically to eliminate the hallucination surface described in Section III: every answer is traceable to a specific retrieved document, with no generative step that could introduce unverifiable content.
- **Anonymous device-ID identity.** LifeLaw has no authentication system. A `crypto.randomUUID()` is generated client-side, persisted in `localStorage`, and used as a stand-in user identifier. This value is stored in the `UserProfile.clerk_user_id` column, a naming choice that anticipates a future migration to Clerk-based authentication without requiring a schema change at that time.
- **Per-source failure isolation.** As described in Section IV, each external API call in `/api/feed` is isolated in its own try/except block so that one dead upstream API degrades the feed rather than breaking it.
- **Hosted embedding inference over local inference.** As described in Section III, embedding computation was moved from a local `sentence-transformers`/PyTorch runtime to the free-tier Hugging Face Inference API after the local runtime's memory footprint caused out-of-memory crashes on a constrained (512MB) deployment instance.

VI. Evaluation

No formal evaluation of retrieval precision, recall, or answer quality has been conducted on LifeLaw to date; the project's own documentation does not report any accuracy metrics, benchmark results, or user study, and none should be inferred. We report this honestly rather than fabricate a number.

A rigorous evaluation of the retrieval engine described in Section III would require, at minimum: (1) a held-out set of representative user questions per rights topic, ideally sourced from real or realistic user queries rather than paraphrases of the corpus documents themselves, to avoid trivially inflating similarity scores; (2) ground-truth relevance judgments mapping each question to the correct source document(s) in `legal_sources`, to compute standard retrieval metrics such as precision@k, recall@k, and mean reciprocal rank; (3) a sensitivity analysis of the cosine-similarity threshold used to decide between "answer" and "no match found," since this threshold directly trades off coverage (how often the system attempts an answer) against precision (how often that answer is actually relevant); and (4) an audit of the regex-based fast-path layer to confirm it does not intercept and mis-route queries that the vector search would have handled correctly, since a fast path that fires on the wrong input is itself a silent failure mode that pure retrieval-metric evaluation of the vector search alone would not catch.

VII. Limitations

Several limitations follow directly from the design choices above. First, because the Q&A engine performs no generation, its usefulness is bounded by the size and coverage of a manually curated, 18-topic rights library and a 13-document legal search index; a question that falls outside curated coverage returns "no match" rather than a degraded-but-useful answer. Second, the system has no authentication: identity is an anonymous, client-generated, `localStorage`-persisted device ID, which means a user's profile and history do not follow them across devices or browsers, and are lost if local storage is cleared. Third, embedding inference depends on a third-party, free-tier hosted API (Hugging Face Inference API), introducing an external latency and availability dependency on every Q&A request rather than a self-hosted model. Fourth, as noted in Section VI, no retrieval-quality evaluation has been performed, so claims about answer relevance are currently unverified by any internal metric. Finally, the platform explicitly disclaims being legal advice or a law firm; jurisdiction filtering and rights education are not a substitute for licensed legal counsel, and the system provides no mechanism to catch cases where curated content has gone stale relative to a jurisdiction's current law.

VIII. Conclusion and Future Work

LifeLaw demonstrates a retrieval-only alternative to generative question answering in a domain where hallucination carries outsized real-world risk. By replacing free-text generation with structured-field assembly over pgvector cosine search, and by making jurisdiction a first-class filter across a live federal/state bill feed, a rights library, and case summaries, the system trades some flexibility of response for a stronger, architecturally enforced guarantee: every answer is traceable to a specific document the system actually retrieved. Future work suggested directly by the system's current gaps includes: conducting the retrieval evaluation outlined in Section VI to

quantify precision/recall and tune the similarity threshold; expanding the curated corpus beyond its current 18 rights topics and 13-document index to reduce ”no match” coverage gaps; migrating from anonymous device-ID identity to authenticated accounts (for which the `clerk_user_id` column has already been reserved) to support cross-device continuity; and evaluating whether a tightly constrained generative layer – for example, one restricted to paraphrasing only the already-retrieved structured fields, with no access to open-ended generation – could improve answer fluency without reopening the hallucination surface that the current architecture is designed to close.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.